实验 04：AI Harness 工具配置及 107 本科生算力平台介绍

Windows 本地协作 · 107 平台 CPU/GPU 实践

本实验先理解模型、API 与 Harness 的分工，在 Windows 上配置 Codex 与 DeepSeek，练习围绕文件推进任务；随后在 107 本科生算力平台提交 CPU/GPU 任务、检查日志、取回结果与释放资源。

阅读路线：第一部分依次学习基础认识、本地配置和决策树协作；第二部分依次完成 CPU 入门、短 GPU 训练和结果管理。已有可用本地 Agent 的同学可从决策树协作开始。

本地 Agent 排错演示可跳过；项目迁移与较大训练、配置管理与工具扩展、DSH 条件与配置放在附录选读。第一次跟练先走通默认路线，遇到问题再查对应的排错表。

从课程仓库首页下载实验 04 学生材料包并完整解压。两个项目位于 `code/`：决策树协作和平台实践。进入对应项目目录后再操作。

自学试用版，2026-10-12。 本实验无需提交代码、截图、结果或报告，不计入评分。文档与配套代码已完成本地检查；干净 Windows、真实模型工具调用与 107 平台 GPU/Slurm 全流程尚待预演，资源和 CUDA 示例需按当前账号与设备核对。

阅读导航

第一部分：AI Harness 工具配置与协作 2
1.1 理解模型与工具协作 2
1.2 在 Windows 上完成配置与验证 5
1.3 用实验 02 代码练习 Agent 协作 9
第二部分：107 本科生算力平台实践 12
2.1 认识平台，提交第一个 CPU 任务 12
2.2 跑通一次短 GPU 训练 14
2.3 取回结果，处理问题并结束任务 19
附录 A：图像分类项目迁移与较大训练（选读） 21
附录 B：配置管理与工具扩展（选读） 23
附录 C：DSH 的条件与配置（选读） 24

第一部分：AI Harness 工具配置与协作

先从网页聊天的经验理解工具协作，再完成 Windows 配置，用实验 02 初始代码观察读取、修改、运行与检查的连续过程。决策树 A→B 练习沿用原先的课程环境。

1.1 理解模型与工具协作

读完本章，应能解释模型怎样生成内容，以及文件读取和命令执行由谁完成。这里先看原理，暂不安装软件。

1.1.1 从网页问答到委托一个小任务

很多同学已经用过网页聊天里的 AI：提出问题，取得解释，继续追问。先沿用这个经验，看一段课程笔记怎样变成一个文件。下面的材料和交互作为阅读用的原理示意；实际操作使用第 1.3 章的决策树项目。

示例文本

```
今天看过 Python 的列表、字典和函数。
函数参数还不熟，需要再看一个例子。
下一次想读决策树程序的 fit 和 predict 方法。
```

问答：取得一段解释

在网页聊天里问“函数参数是什么意思”，AI 可以生成解释和例子。你负责阅读、判断它是否回答了问题，再提出下一次问题。

代写：生成你需要的内容

把这段笔记贴进聊天，要求：“按已学内容和待复习内容整理成 Markdown，保留原有信息。”AI 可以给出整理后的文字。在纯文字协作中，你把结果复制到编辑器，保存文件，再把需要调整的部分贴回聊天。

这个过程已经能省下写作工作。文件的保存、结果检查和下一步材料传递，仍由你连接起来。

委托工具操作：围绕文件继续推进

假设笔记已保存在练习文件夹的 `notes.txt` 中。向具备文件工具的 agent 提出同一个目标：

示例文本

```
请读取当前练习目录中的 notes.txt，
按已学内容和待复习内容整理成 study-notes.md。
保留原有信息，操作范围仅限这两个文件。
完成后读取生成文件，核对内容，并说明实际结果。
```

在工具和权限支持时，系统可以读取原文件、生成整理内容、写入文件，再读取结果检查。你还可以继续要求“把待复习内容改成简短列表”，让它依据当前文件修订。

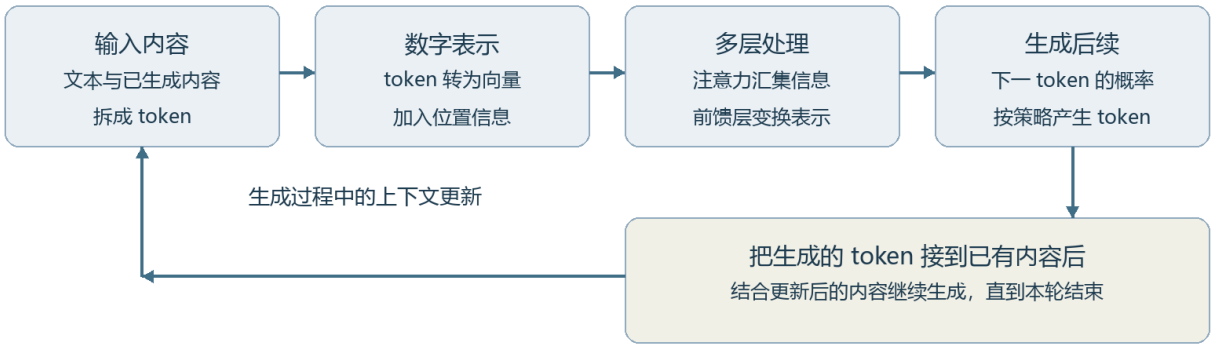
区别在于**由谁把步骤和反馈连接起来**：纯文字协作中，学生传递材料并执行操作；agent 的运行程序可以承担其中一部分。它能推进到哪一步，取决于模型、工具、权限和任务是否清楚。无法确定或需要额外权限时，应停下来说明或请你判断。

网页是操作入口。有些网页产品同样提供工具和任务执行能力；桌面应用也可能只是聊天界面。判断协作方式，要看它能否实际调用工具、取得反馈并据此继续工作。接下来先理解模型在其中做什么。

1.1.2 模型怎样生成内容：浅看 Transformer

Transformer 是一类神经网络结构。这里用它理解生成式语言模型的基本过程，先记住“输入如何变成数字、怎样利用上下文、怎样接着生成”即可。不同模型的具体结构会有变化。

从输入到后续内容：生成过程示意



1. **把输入表示为数字。** 文本先拆成 token（词元），每个 token 再转换为一组数字，也就是向量；同时加入顺序或位置信息。Token 的边界由具体模型的分词方式决定，不一定对应一个字或一个完整词。
2. **结合上下文处理。**注意力机制根据当前表示之间的关联分配权重、汇集信息。例如理解“把笔记保存成文件，再修改它”时，前面的内容会影响后续解释。这个例子只表示上下文关系，没有展示某个模型的实测注意力权重。前馈层等部件继续变换表示，这样的处理会重复多层。
3. **继续生成。**模型计算下一 token 的概率，按照生成策略产生后续内容，再结合已经生成的部分继续。多次重复后，得到一段解释、整理后的笔记或代码。Transformer 原论文介绍了这些部件。

训练与推理分别做什么？ 训练用数据调整模型内部的参数；我们日常调用已训练模型、让它处理输入的过程叫推理。你在当前会话补充要求，通常是在改变它下一次看到的上下文，并不意味着即时重新训练了模型。应用也可以另外保存会话或资料，保存方式由应用决定。

模型生成的内容需要检查。给出的事实、整理结果和操作建议是否正确，要靠来源或实际结果核对。现在可以继续问：如果模型生成的是一个“读取文件”的请求，谁真正去读取？

1.1.3 工具调用与模型、API、Harness 的分工

模型除了给出面向人的回答，也可以在相应能力、协议和工具说明支持下，生成一个工具调用请求。运行程序检查并执行这个请求，把工具结果送回模型，模型再决定是继续请求还是给出回答。工具调用流程描述了这个反馈过程。

刚才的笔记整理任务可以形成这样的循环。以下是步骤示意，具体工具名称和调用格式随软件变化：

示例文本

学生给出目标与范围
→ 模型请求读取 `notes.txt`
→ 程序读取文件，把内容返回模型
→ 模型生成整理内容并请求写入 `study-notes.md`
→ 程序写入并返回结果
→ 模型结合结果继续检查或回复
→ 学生打开文件，检查并提出修订

“写入成功”只说明工具完成了对应操作，内容是否完整仍需检查。这里所说的主动推进，是在给定目标和获准范围内选择后续步骤；是否需要补充信息、请求授权或停下来，由任务状态和运行程序共同决定。

分清模型、API 服务与 harness

之后对助手说“只补全信息熵方法，再运行几个小样例”，也会涉及同样的几个部分。

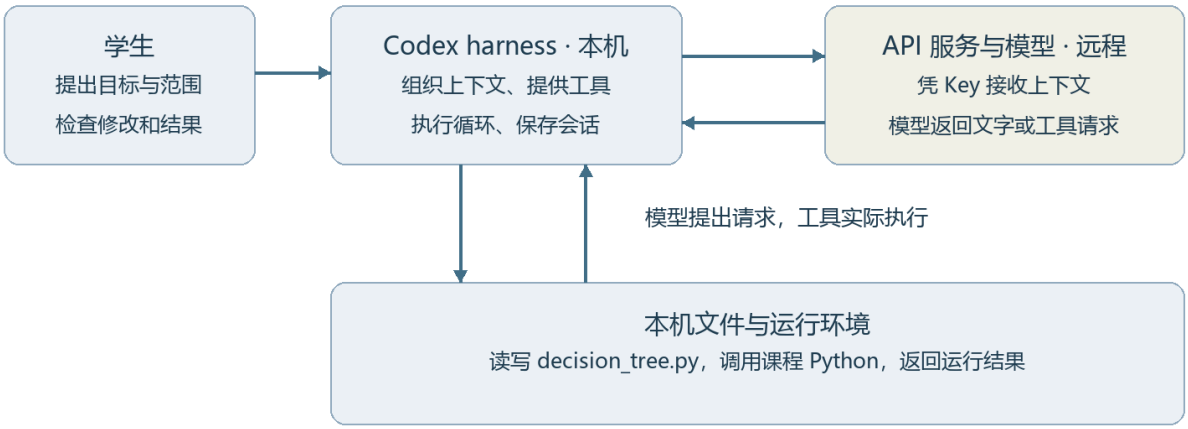
模型根据收到的上下文生成文字或工具调用请求。它负责理解要求、提出方案和判断下一步。DeepSeek 的模型属于这一层。

API 服务把模型推理能力提供给客户端。API 是程序之间交流的接口；服务地址决定请求发到哪里，API Key 用来识别调用者和权限。相同名称的模型由不同平台提供时，部署、速度、额度和价格可以不同。

Harness 可以理解为 agent 的运行框架。它组织模型看到的上下文，提供工具，执行获准的操作，把结果送回模型，并保存会话。Codex 属于这一层；本教程通过桌面应用使用它。

Agent 是把模型、工具和运行循环组合起来，围绕一个目标继续推进任务的系统。模型可以请求读取文件，但真正读取文件的是 harness 提供的工具；能否执行还取决于当前权限。

一次 agent 协作涉及的层次



图中的远程推理发生在 API 服务端。文件读取、代码修改和 Python 命令通常在你的电脑上执行。运行 Python 使用本机 CPU 和课程环境，无需在自己的电脑部署 DeepSeek 模型。被选入上下文的代码和工具输出会发送给 API 服务，因此练习先使用公开课程材料。

Codex 这个名称也可能出现在模型产品中。本教程所说的“Codex harness”指运行任务的软件。连接 DeepSeek 后，任务所用模型由实际 API 配置决定。模型自称什么不能替代配置核对。

以后换工具时，仍可以沿用这个思路：**我用哪个运行框架、连接哪个服务、调用哪个模型、允许哪些工具、怎样检查结果？**

读完后自问：如果模型提出“读取文件”，谁执行读取、在哪里执行、结果怎样返回？能说清这个循环，就可以进入配置。

1.2 在 Windows 上完成配置与验证

本章先看费用和接入条件，再选择服务。默认操作路线为 Codex + DeepSeek 官方 API；已有学校 Key 或桌面入口受限的同学，先看“学校 API 与回退路径”，选择对应客户端和服务地址。

1.2.1 费用与国内网络

软件费用与模型费用分别看

本教程走 DeepSeek 官方 API 的按量调用路径。软件或配置管理器免费，并不意味着模型调用没有费用；桌面账号的订阅额度也不能直接当作 DeepSeek API 余额。平台赠送额度、学校授权额度与自己的充值余额，要分别查看。

前面“浅看 Transformer”中见过的 token 也是 API 常用的计费单位，不能固定换算成中文字符数。费用通常分别统计：

- **输入**：你发出的要求，以及程序随请求传入的指令、代码、历史消息和工具结果。
- **缓存输入**：服务端复用了此前处理过的输入前缀，可能采用较低单价；仍可能计费。
- **输出**：模型生成的内容；推理内容的计费口径也应查看服务商说明。

一次 agent 任务往往包含多次请求。读取代码、执行检查、看到失败后再修改，会让新的工具结果和历史进入后续上下文。会话界面里发了一条消息，账单上可能记录了多次模型调用。

下面是 **2026-10-12 核对的 `deepseek-flash` 人民币价格示例**，单位为元／每百万 token。充值前再看官方中文价格页，以账户实际结算为准。

类型	低谷价格	高峰价格
输入，缓存命中	0.02	0.04
输入，缓存未命中	1.00	2.00
输出	4.00	8.00

例如一次请求在低谷时段消耗 10,000 个未缓存输入 token、20,000 个缓存输入 token 和 3,000 个输出 token：

示例文本

```
费用 = (10000 × 1.00 + 20000 × 0.02 + 3000 × 4.00) / 1000000
     = 0.0224 元
```

这是算账示例，不能用它预测整次练习的总费用。先完成一次小范围的决策树协作任务，再查看用量与余额变化；决策树 A/B 可按需要选练。任务停在不明原因的重试中时，先停止并检查错误，避免持续重复请求。

国内网络可行性要分环节判断

DeepSeek 官方已提供 Codex 接入方式，并原生支持 Codex 使用的 Responses API。这说明接入有明确依据。完整配置还涉及客户端下载安装、脚本下载、密钥申请、模型调用和文件工具执行，任何一个环节失败都需要分别排查。

本教程按国内 API 路径编写；“全程无代理也一定成功”需要具体网络和客户端版本的验证，本教程不作这一保证。如果 Codex 安装或启动入口不顺，先看学校 API 与回退路径；DSH 仅在独立实验环境内选读试用。不要为了赶进度同时安装多款管理器或反复重写配置。

1.2.2 学校 API 与回退路径

推荐尝试学校提供的免费 API 资源

打开中国科大大模型公共服务平台，按申请流程通过统一身份认证登录，创建用途明确的项目，再申请 API Key。免费资源的可用模型、额度与有效期以实际授权为准。

学校 Key 要连接学校网关；它不能直接填入固定连接 DeepSeek 官网的配置脚本。学校入口、模型服务地址和模型 ID 的用途不同，复制当前可用模型的准确 ID 即可。学校已有图文入门指南，可直接参考对应客户端章节。

校内共享服务的实际速度可能不及官方或较大平台的 API，尤其在高并发、长上下文或复杂推理时。TPS 是每秒生成的 token 数，还应分别观察首 token 等待时间和总耗时。当前没有足够的统一对比实测，不能仅从硬件规格给出快慢结论；先用短任务感受响应，再决定是否执行完整 B 阶段。

选择一条适合自己的回退路径

情况	可以尝试的路径
Codex 桌面下载或启动不顺	可看 CLI 补充入口；DSH 桌面版仅在独立实验环境中试用
有学校 Key，希望图形化操作	参考学校指南配置 WorkBuddy；DSH 的条件与配置见附录 C
主要处理办公文件	参考学校指南中的 WorkBuddy 配置方式
已熟悉终端，桌面入口受限	使用 Codex CLI 作为补充入口

学校指南中的社区 DSH Desktop 与 DeepSeek 官方桌面发行版不是同一个安装包，设置界面和数据目录可能不同。跟做时以所选发行版的说明为准。

学校常见 Chat Completions 基础地址为 `https://api.llm.ustc.edu.cn/v1`；WorkBuddy 的示例字段要求完整地址 `https://api.llm.ustc.edu.cn/v1/chat/completions`。Codex 需要 Responses 接口，接学校服务前要确认开通范围和适用模型。换了工具，仍然可以沿用决策树项目与同一任务目标，按下文检查读取、修改和运行是否实际完成。

1.2.3 连接 Codex 与 DeepSeek

安装与配置视频参考：Codex 原生直连 DeepSeek：官方脚本接入与安装使用教程（B 站）。视频采用官方脚本接入路线，可结合下文观察下载安装、获取 API Key 和脚本配置的操作。视频中的客户端界面和模型名称可能随版本变化，按本讲义与当前官方说明填写。安装部分采用这份外部视频。

步骤 1：准备客户端和 API Key

1. 从官方 Windows 桌面说明进入下载与安装流程。新版本可能以 ChatGPT 桌面应用承载 Codex，界面名称以官方下载页为准。
2. 安装后启动一次。DeepSeek 脚本需要已有的用户配置目录；默认位置是 `%USERPROFILE%\.codex`。
3. 打开 DeepSeek API 平台，申请自己的 API Key，并查看当前余额与计费规则。已有学校 API Key 的同学先看学校 API 与回退路径，学校 Key 与 DeepSeek 官方 Key 对应不同服务地址。

客户端首次启动可能有登录或模式选择。若仍无法进入自定义 API 路径，保留界面提示，参考学校 API 与回退路径或 CLI 补充入口；不要把未完成的登录步骤误判为模型请求失败。

API Key 是调用凭据。把它保存在可信客户端的密钥设置中，或按官方接入步骤处理。不要放进聊天提示词、公开代码和共享截图。泄露后应在对应平台撤销旧 Key，再创建新的 Key。

步骤 2：运行官方配置脚本

先完全退出桌面应用，再打开普通 PowerShell。以下命令来自 DeepSeek 官方 Codex 接入文档，会下载并执行配置脚本：

PowerShell • Windows

```
irm https://cdn.deepseek.com/api-docs/codex-deepseek-setup-en.ps1 | iex
```

按当前菜单选择 `1`，使用 `deepseek-flash`，然后填写自己的 DeepSeek API Key。当前脚本的输入框不遮住密钥字符，操作时避免屏幕共享和录屏。

脚本会备份旧配置，写入模型目录和连接设置，并提示重新启动应用。它不会安装客户端，也不会真正发送一条模型请求来验证 Key。配置写入成功后，继续步骤 3，验证一次文字调用。

已有自定义配置的同学先阅读脚本变更说明。当前脚本会处理冲突设置，包括部分 profile 和指令覆盖项。恢复选项会还原安装前的配置，安装后新增的配置需要另行保存。Windows 脚本源码可以直接下载查看。

步骤 3：先做文字问答，再做工具操作

重新打开桌面应用，确认当前模型显示与 DeepSeek 配置一致。新建会话并发送：

示例文本

```
请用三句话解释信息熵。这次只回答文字，不读取或修改文件。
```

正常得到回答，说明至少完成了一次文字调用。回到 DeepSeek 平台查看用量变化，确认请求记在预期账户中。随后按决策树协作练习选择练习副本，再要求读取代码和运行自查。文字问答成功与文件工具成功要分开确认。

如果模型选择或界面名称与教程不同，优先检查实际提供方和模型 ID；完全退出应用后再打开，关闭窗口可能仍让程序留在后台。

步骤 4：看懂脚本写了什么

关闭客户端后，可以从资源管理器打开 `%USERPROFILE%\.codex`。不要把含密钥的配置全文发给助手或他人。重点看四个信息：

信息	本教程的含义
`base_url`	DeepSeek 官方服务入口，当前脚本固定为 `https://api.deepseek.com/`
`wire_api`	请求协议，当前使用 `responses`
API Key	当前脚本以 `experimental_bearer_token` 写入本机配置；应遮住它
`model`	此次选择的模型 ID，主线为 `deepseek-flash`

`config.toml` 保存连接和运行设置；`models.json` 告诉客户端模型的上下文、推理档位、工具格式等能力信息。只改一个模型名称，未必能完成适配。

“OpenAI 兼容”描述的是接口格式；实际模型提供方由服务地址和权限决定。Chat Completions 与 Responses 是不同接口，仅写“兼容 OpenAI”仍不足以判断 Codex 能否接入。Codex 配置说明介绍了这些字段。

1.2.4 配置与调用问题速查

现象	先检查什么
脚本提示配置目录不存在	客户端是否启动过；是否用了自定义配置目录
下载脚本失败	官方地址是否能打开；网络、TLS 或学校网络策略；先保留完整错误
401 / 403	Key 是否属于该服务；权限、有效期、余额与平台错误说明
404 / 参数不兼容	协议与地址是否匹配；是否给要求基础地址的字段填了完整接口
配置后仍显示旧模型	是否彻底退出应用再打开；当前会话是否加载了新配置
能回答文字，但不能改文件	工作目录、文件权限、工具调用兼容性与授权提示
agent 的 Python 命令失败	它使用的解释器路径是否与课程终端一致
初始程序输出 None	T1–T7 尚未补全；这是初始骨架的预期行为
回答很慢或反复重试	区分等待、限流与错误；停止重复请求后查看服务状态和用量

求助时给出客户端版本、服务提供方、模型 ID、操作步骤和完整错误；遮住 API Key。描述“问答成功、读取成功、运行失败”比只说“不能用”更容易定位。

进入协作练习前：确认一次文字请求确实发送到预期服务，并能查看相应账户用量。下一章会继续核对文件读取和结果保存；Python 执行属于决策树选练。需要维护多套配置或使用 CLI 时，再看附录 B。

1.3 用实验 02 代码练习 Agent 协作

使用实验 02 的决策树初始骨架，先限定 T1 观察一次完整协作，再扩展到 T2–T7。环境沿用实验 01/02 的课程 Python，本实验不要求另交作业；已熟悉工具协作的同学可直接进入平台部分。

1.3.1 准备决策树副本与课程环境

完整解压实验 04 材料包，将其中的 `code/decision-tree-practice/` 复制到自己的练习位置，例如 `文档\agent-lab2`。保留下列文件：

示例文本

```
agent-lab2/
  AGENTS.md
  README.md
  decision_tree.py
  checks.py
```

在桌面应用中选择这个文件夹作为工作目录，再新建会话。练习目录是一份副本，可以放心比较改动，也方便重新开始。

打开自己的课程终端，先进入这份 `agent-lab2/` 练习副本的根目录，确认能看到 `decision_tree.py` 和 `checks.py`；环境与实验 02 一致：沿用实验 01 创建的 `ustc-ai-experiments-26fall-B`，课程版本为 Python 3.11。随后激活它，核对版本和解释器并运行：

PowerShell • Windows

```
conda activate ustc-ai-experiments-26fall-B
python --version
python -c "import sys; print(sys.executable)"
python decision_tree.py
```

前三条沿用实验 01/02 的环境，版本应为 `3.11.x`，解释器应位于该课程环境中。如果 `conda` 在普通 PowerShell 中不可用，先在已有的 Miniforge Prompt 激活环境，确认解释器位置。将打印出的 `python.exe` 完整路径告诉 agent；桌面应用启动的命令进程可能未继承你刚刚激活的终端环境。

初始代码的 T1–T7 方法返回 `None`。演示出现 `None` 是占位结果，按步骤补全后会改变。程序只使用 Python 标准库，无需为这个练习安装 NumPy、scikit-learn 或额外的决策树库。

先给 agent 一个读取任务：

示例文本

```
请阅读 README.md、AGENTS.md 和 decision_tree.py。
说明数据怎么进入程序、训练和预测分别调用哪些方法，
并列出 T1–T7 的依赖关系。先只解释，不修改文件。
```

展开工具记录，确认它读取了正确文件。仅根据文件名猜测代码内容，与实际读取代码有不同的证据。

1.3.2 A 阶段：只完成 T1，观察一次完整协作

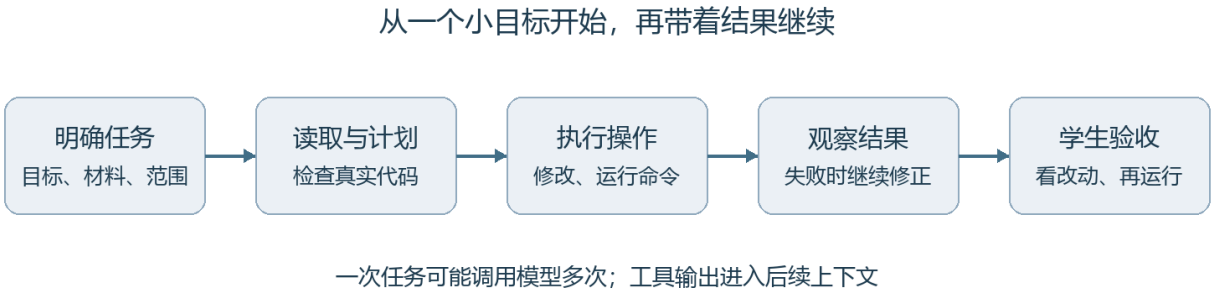
这次只补全 `_entropy`。你可以先手算两个结果：`[0, 1]` 的信息熵为 1，`[1, 1]` 的信息熵为 0。

把自己的解释器路径一并提供给 agent，然后发送：

示例文本

```
请只补全 decision_tree.py 的 T1: _entropy。
先简要解释思路，再修改这个方法，保持其余 TODO 不变。
遵守空列表返回 0.0 的约定，并跳过概率为 0 的项。
使用我指定的课程 Python 运行 python checks.py --stage a。
最后说明改了什么、实际运行了什么、哪些任务仍未完成。
```

执行命令或修改文件时，客户端可能请求授权。看清命令、目标文件夹和预期动作，再允许它执行。项目里的 `AGENTS.md` 能说明约定；实际访问控制仍由客户端权限和操作系统决定。



完成后自己检查：
1. 打开修改前后对比，确认改动集中在 T1。
2. 在自己的课程终端运行 `python checks.py --stage a`，预期为 `5/5 passed`。
3. 再运行 `python decision_tree.py`。信息熵应有数值，其余待补方法仍可能输出 `None`。
4. 追问：“为什么纯标签的信息熵是 0？为什么不能计算 `log2(0)`？”

自查脚本包含空输入、两种纯标签、均衡标签和不均衡标签。它让你看到具体输入和预期输出，模型口头说“测试通过”不能替代实际执行记录。

如果 agent 顺便完成了其他 TODO，可以要求：“这次范围仅限 T1，请展示额外改动，并撤回那些改动。”修改前已有的内容要先区分，避免把自己的代码一起撤回。

1.3.3 B 阶段：把同样的方法用于 T2–T7

已有的 T1 保留。把任务扩展到完整实现，先让 agent 制定步骤：

示例文本

```
请基于当前 T1 实现，完成 T2–T7。
先按方法依赖给出简短计划，然后分阶段实现并运行验证。
保持原始特征列编号，遵守平票、零增益继续划分和未知分支回退规则。
不要修改内置数据、方法签名、题目约定或 checks.py。
使用我指定的课程 Python，最后运行 checks.py --stage b
和 decision_tree.py。汇报实际结果；失败时说明原因并继续修正。
```

可以按 T2–T4、T5、T6–T7 三段观察进展。关键不在于固定采用哪种代码风格，而在于每一段完成后都有可检查的结果。

B 阶段自己运行：

PowerShell • Windows

```
python checks.py --stage b
python decision_tree.py
```

检查内容包括信息增益加权、增益并列顺序、递归终止条件、零增益的 XOR 小例子、原始列编号、未知分支和批量预测顺序。完整西瓜数据应选择“纹理”（列编号 3）作为根节点，信息增益约为 `0.380592`。

遇到失败时，把**失败项目、完整错误和当前改动**交给 agent，例如：

示例文本

```
自查中的未知分支项目失败，预期返回当前节点多数类。
请先读取相关实现并解释失败原因，再做最小修正。
修正后重新运行该项目和完整 B 阶段检查，保持 checks.py 不变。
```

验收时看三类证据：

- **修改范围**：数据和检查标准保持一致，程序实现符合题目约定。
- **实际执行**：能看到运行命令、具体输出和失败后的修正。
- **自己的理解**：可以让 agent 用小例子解释某个分支，并检查解释与代码是否一致。

检查通过说明这些样例上的行为符合约定，不是对所有输入的证明。本实验在同一批 17 条记录上训练并回代预测；回代结果不能代表独立测试集上的泛化表现。

1.3.4 归纳任务要求与项目约定

把目标、范围和检查方法说清楚

任务请求要说清结果、材料、修改范围和检查方法。例如“阅读这两个文件，只修改这个方法，用这几个输入检查，并说明仍未验证的部分”。遇到陌生问题先请求解释和计划，范围明确后再让 agent 执行。这套方式也适用于资料整理和脚本改进。

用一份短的 AGENTS.md 保存重复约定

练习中的 `AGENTS.md` 保存了先读说明、采用课程环境、遵守实验约定和运行自查等重复要求。

项目文件不会增加模型原本没有的能力，也不能代替权限设置。规则应当短、具体、能检查。例如“保持内置数据不变”比“写出完美代码”更容易落实。

了解读取位置与覆盖关系，可看 AGENTS.md 官方说明。新手先在练习根目录用一份即可。

同一个任务继续聊，换目标时带上交接

当前任务还在调试时，继续原会话能保留上下文。目标明显变化或会话过长时，可以新开会话，带上文件路径、已完成内容、最近的检查结果和下一步。

例如：“T1–T7 已完成，自查通过；当前要解释未知分支行为，先读 decision_tree.py。”最终状态应以实际文件和运行结果为准。保存文件才能保留成果，聊天记录不能替代项目文件。

进入平台实践前：确认练习副本中的内置数据和 checks.py 保持原样，能查看实际代码差异和运行结果，并说明工具怎样检查结果。决策树 A/B 自查分别为 `5/5`、`16/16 passed`。这些结果自行保留即可。

第二部分：107 本科生算力平台实践

先完成网页 CPU 小任务，再配置独立环境、准备 CIFAR-10 + ResNet-18 并用脚本提交短 GPU 训练。本地 Agent 协助修改代码，学生在平台执行，取回真实日志和结果。较大训练放在附录 A。

2.1 认识平台，提交第一个 CPU 任务

本章先分清本地、登录节点与计算节点，再运行一个仅依赖 Python 标准库的小程序，确认上传、提交、查看日志和下载结果的链路。

2.1.1 什么任务适合交给平台

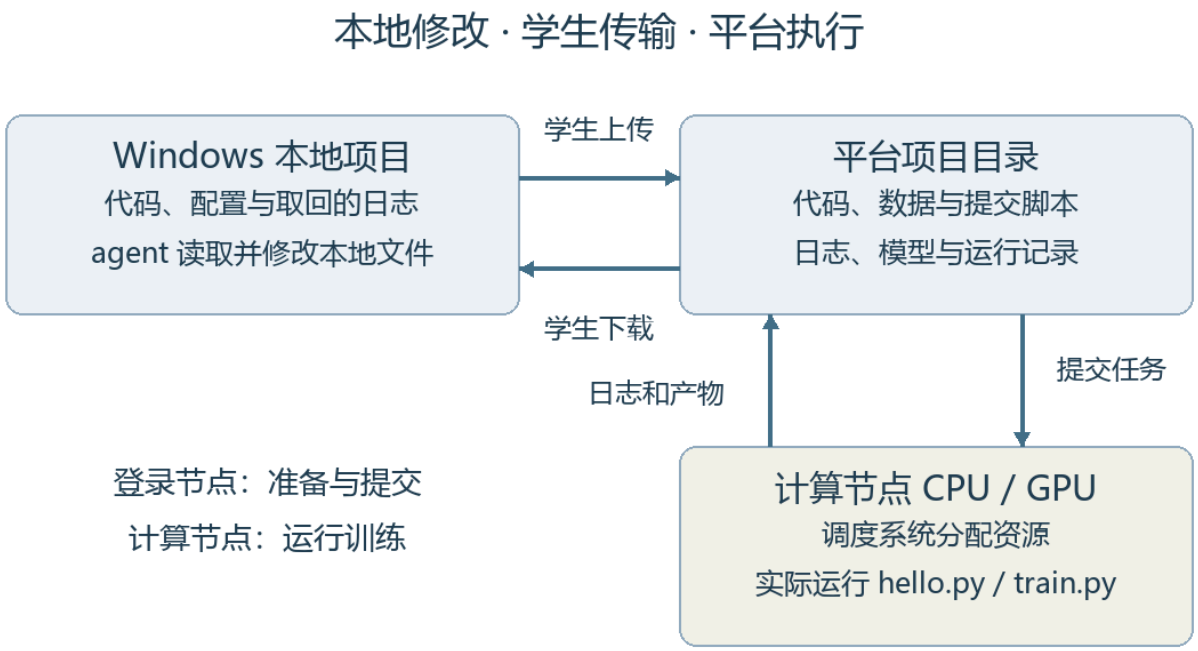
你已经能够在自己的电脑上运行 Python 程序。对于决策树或小型表格分类练习，本地 CPU 通常就够用；图像模型反复训练时，计算量、内存和运行时间可能更突出。平台把学校管理的 CPU、GPU 和内存提供给计算任务，让个人电脑条件不同的同学也能尝试较大的实验。

CPU 处理通用运算、数据读取和调度；GPU 擅长执行大量相似运算，例如神经网络中的张量计算。程序需要把模型和数据放到 GPU，申请到 GPU 本身不会让任意 Python 程序自动变快。主内存供 CPU 侧程序使用，显存供 GPU 使用，二者的不足需要分别排查。

平台也适合 CPU 长任务、数值模拟和批量实验。使用它还需要配置环境、传输文件和等待资源。判断是否有帮助，要看完整任务的完成时间以及能否稳定复现，而非只比较设备名称。平台概览

模型 API 的费用与算力平台资源分别管理。第一部分使用 API 调用模型；本部分在平台运行自己的训练程序。平台账号的授权、额度、有效期及是否计费，以当前账号说明为准。

2.1.2 本地、登录节点与计算节点



关系示意，节点与界面以平台预演为准

- **Windows 本地**：保存代码和取回的日志，本地 agent 在这里读取、修改文件。
- **登录节点**：整理文件、编辑脚本、配置环境、提交任务。训练通过资源申请进入计算节点。
- **计算节点**：调度系统分配 CPU/GPU 后，真正运行任务。

· **共享工作目录**：让提交入口与计算任务访问代码、数据和结果；实际共享范围由平台确认。Slurm 是任务调度系统。你告诉它要运行什么、需要多少资源和最多运行多久，它安排执行。**提交成功得到任务 ID，意味着任务已经登记；排队和实际运行是后续状态。**

平台的 Web Shell、计算节点 Shell 和 VS Code 应用终端可能在不同节点，资源生命周期也不同。每一步先看本教程标出的执行位置。学生仍在 Windows 浏览器操作，终端里的 Linux 命令由平台执行。

2.1.3 上传、提交与检查 CPU 任务

步骤 1：准备与上传代码

打开平台入口，通过统一身份认证方式登录。找到“文件管理”“作业”和“登录集群”等入口。下面的按钮名称来自平台文档，界面随版本变化时查看当前说明。GUI 说明

完整解压实验 04 材料包，进入 `code/compute-practice/`，能看到 `hello.py`、`train.py`、`configs/` 和 `scripts/`。包里不含数据、环境和训练结果。

通过文件管理把解压后的 `compute-practice/` 文件夹及其内容上传到平台用户目录下的 `projects/`。如果上传的是完整 ZIP，先解压，再取其中的 `compute-practice/` 放到这个位置。后文示例项目根目录为 `~/projects/compute-practice`；打开它应直接看到 `hello.py`、`requirements.txt`、`configs/` 和 `scripts/`，不要再嵌套一层同名目录。这里的“作业”指平台计算任务，本实验无需课程作业提交。

步骤 2：填写网页任务字段

进入“作业 → 提交作业”，把工作目录设为项目根目录。在命令区域填写：

Bash · 平台终端

```
python hello.py
```

第一次选择 **CPU 任务，GPU 数为 0**，使用当前账号允许的较小 CPU/内存配置和短时限。把标准输出、错误日志位置记下来；若填写相对日志目录，先在文件管理中创建它。

这里的 Python 是平台已有解释器，hello 程序仅用标准库。若当前页面要求 `python3`，相应改为 `python3 hello.py`。这一步先确认入口、文件和任务流程。

提交后记录任务 ID，在作业列表中查看状态。开始运行后，输出应包含：

示例文本

```
hello from computing platform
```

还会显示解释器、当前目录和节点。程序保存 `outputs/hello/result.json`。打开该文件并下载到本地，确认内容与这次运行的日志一致。此处固定 hello 结果允许重复写入，后面的训练会为不同运行保留独立目录。

检查点：看到任务 ID、实际运行日志和下载的结果文件。仅看到“已提交”还不算跑通。

2.2 跑通一次短 GPU 训练

沿用刚才的项目根目录，依次准备环境、数据和提交脚本。本章的完成标准是实际 CUDA 运算及可读取的产物，短训练的准确率不作为完成标准。

2.2.1 分步配置项目环境

以下命令在**平台登录集群的网页终端**中执行。先进入刚才的项目根目录；把第一行示例路径改为实际项目位置：

Bash · 平台终端

```
cd ~/projects/compute-practice
pwd
ls
```

`pwd` 是当前目录，`ls` 列出文件。应看到 `requirements.txt`、`configs/`、`scripts/`。路径不对时先修正，再安装和提交。

步骤 1：检查已有 conda；没有时再安装

Bash · 平台终端

```
command -v conda
python --version
```

已有可用 conda 时跳到步骤 2，创建独立 Python 环境。若没有，可以在用户目录安装 Miniforge。以下是 Linux x86_64 示例，先用 `uname -m` 核对平台架构，再从学校镜像下载安装器。中科大镜像说明

Bash · 平台终端

```
uname -m
mkdir -p ~/downloads
cd ~/downloads
compute_mirror=https://mirrors.ustc.edu.cn/github-release
compute_release="$compute_mirror/conda-forge/miniforge/LatestRelease"
compute_installer=Miniforge3-26.7.2-0-Linux-x86_64.sh
wget "$compute_release/$compute_installer"
bash "$compute_installer" -b -p "$HOME/miniforge3"
source "$HOME/miniforge3/etc/profile.d/conda.sh"
cd ~/projects/compute-practice
```

安装器名称按 2026-10-11 镜像目录写出；目录更新后选择对应架构的安装器。已有 `~/miniforge3` 时不要直接重复安装；检查现有环境或选择新的安装位置。这里没有改写全局软件源，`source` 为当前终端加载 conda。

步骤 2：创建独立 Python 环境

环境管理器负责组织解释器和依赖，项目环境才是这次训练使用的软件集合。给项目建一个单独的环境，避免与别的实验互相影响：

Bash · 平台终端

```
conda create --prefix "$HOME/.conda/envs/compute-resnet" \
  --override-channels \
  -c https://mirrors.ustc.edu.cn/anaconda/cloud/conda-forge \
  python=3.11 pip -y
conda activate "$HOME/.conda/envs/compute-resnet"
which python
python --version
```

`which python` 应指向 `compute-resnet/bin/python`。路径已存在时先确认是不是本项目环境，不要为了重装删除已有环境。

步骤 3：选择 PyTorch 构建并安装

本教程使用 `torch 2.7.1 + torchvision 0.22.1` 作为固定参考组合，便于复核。它是官方仍提供的历史版本组合，不是“最新版”的承诺；请按平台当前环境说明核对兼容性。官方安装组合

PyTorch wheel 的 CUDA 构建要与目标 GPU 和平台驱动兼容，不能根据登录节点没有 GPU 就选择 CPU 包。以下 `cu128` 是待平台核验的候选示例，可用于评估 Blackwell 支持；平台驱动由维护者管理。PyTorch Blackwell 支持

核对平台建议后，在已激活的环境中执行对应安装命令。候选 `cu128` 示例：

Bash · 平台终端

```
python -m pip install torch==2.7.1 torchvision==0.22.1 \
  --index-url https://download.pytorch.org/whl/cu128
python -m pip install -r requirements.txt \
  --index-url https://mirrors.ustc.edu.cn/pypi/web/simple
python - <<'PY'
import torch, torchvision
print(torch.__version__, torchvision.__version__, torch.version.cuda)
PY
```

普通依赖从 PyPI 镜像获取，PyTorch 构建从指定 wheel 索引获取，两者作用不同。下载失败时保留命令和错误，查询平台推荐的下载或离线包方案；这些来源在纯国内网络下的可达性取决于实际网络，尚未完成统一预演。

导入成功证明依赖能加载，GPU 运算还要在实际申请到 GPU 的任务中验证。登录节点上的 `torch.cuda.is_available()` 为 False，不能单独判定项目环境安装失败。

步骤 4：后续复用同样的步骤

首次理解分步配置后，可以在新环境位置使用复用脚本：

Bash · 平台终端

```
bash scripts/setup-env.sh cu128 "$HOME/.conda/envs/compute-resnet-next"
```

它接受 `cpu`、`cu126`、`cu128`，创建指定的新环境，不覆盖现有环境路径。GPU 构建仍须先核验。提交脚本用环境内 Python 的完整路径，等效地选择同一解释器；它不依赖刚才网页终端的激活状态。

2.2.2 准备 CIFAR-10 与短训练配置

CIFAR-10 包含十类 32×32 彩色图像。程序用 ResNet-18 处理图像并输出十类得分，从随机初始化开始训练；把第一层和池化调整到小图像输入。训练代码与数据准备脚本已提供，数据按下文下载；当前重点是让程序在正确位置运行。数据来源、模型说明

步骤 1：下载、校验、解包

首次仍需要完整 CIFAR-10 Python 原始包，约 163 MiB；少样本参数只减少训练用量。可以从原始数据页下载。网络慢时优先在本地下载后上传，训练脚本不在线下载数据。

在 **Windows 本地 PowerShell**，进入解压后的 `compute-practice/` 项目根目录，再下载与校验：

PowerShell · Windows

```
Invoke-WebRequest `
  -Uri 'https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz' `
  -OutFile '.\cifar-10-python.tar.gz'
Get-FileHash -Algorithm MD5 -LiteralPath '.\cifar-10-python.tar.gz'
```

原始 MD5 为 `c58f30108f718f92721af3b95e74349a`，大小写不影响对比。该值用于对照原始包和传输完整性，下载来源也要核对。

把压缩包上传到**平台项目根目录**。在平台已配置的终端中解包：

Bash · 平台终端

```
python prepare_data.py --archive cifar-10-python.tar.gz --data-dir data
python prepare_data.py --data-dir data --check-only
```

应显示 `verified CIFAR-10`。程序只提取预期数据文件，并校验包内数据，不会改写已经存在的数据目录。若已有目录但校验失败，先检查上传是否完整，选一个新的数据目录恢复，不把不完整文件当成可用数据。

目录关系：

示例文本

```
compute-practice/
  hello.py                    # CPU 入门
  train.py                    # 主线与选读共用训练入口
  prepare_data.py              # 原始包校验和解包
  inspect_run.py               # 读取结果和模型
  configs/smoke.json           # 256 训练样本、8 步的短训练配置
  configs/train.json           # 较大训练候选配置
  scripts/                    # 环境与任务脚本
  data/cifar-10-batches-py/ # 已解包数据
  logs/                       # Slurm 日志，提交前创建
  outputs/                    # 按运行区分结果
```

步骤 2：读懂短训练配置

`configs/smoke.json` 使用 256 个训练样本、128 个验证样本、batch 32、最多 8 次参数更新。样本数决定看多少数据，batch 是每次更新用的样本数，`max_steps` 限制整个任务更新次数；`0` 表示由 epoch 决定、不额外限制。

程序从官方训练集固定划出 45000/5000 的训练/验证池，短训练取其中的小部分，官方测试集不参与。这样的短任务用于核验流程，不以准确率高低为完成标准。

参数是短训练的参考起点，不代表平台已测定的最优配置或固定耗时。主线和选读使用同一脚本、固定种子和划分，主要改变运行配置。

2.2.3 用 sbatch 提交并检查训练

步骤 1：对照网页字段阅读脚本

打开 `scripts/smoke.sbatch`，对应查看这些字段：

网页里的含义	脚本字段	示例
分区 / 资源分组	`--partition`	`Students`
资源规则	`--qos`	`qos_stu_default`
GPU 数量或类型	`--gres`	`gpu:1`
CPU 核心数	`--cpus-per-task`	`2`
CPU 主内存	`--mem`	`8G`
最长运行时间	`--time`	`00:15:00`
输出和错误日志	`--output` / `--error`	`logs/%x-%j.out/.err`

这些值来自示例，提交前确认你的账号可用范围及 GPU 申请写法。`#SBATCH` 行由 Slurm 解析，shell 变量不会自动展开；`%x`、`%j` 是作业名与任务 ID 的替换规则。Slurm sbatch

步骤 2：提交并观察

在**平台提交终端，项目根目录**执行：

Bash · 平台终端

```
export COMPUTE_ENV_PREFIX="$HOME/.conda/envs/compute-resnet"
bash scripts/submit.sh smoke
squeue -u "$USER"
```

复用提交入口先建立日志目录，并让 Slurm 在项目根目录启动。任务用 `COMPUTE_ENV_PREFIX/bin/python`，因此环境位置不一致时先修正该变量。复制终端激活命令到新窗口，不等于批处理已选择同一环境。

平台需要指定 GPU 型号或不同 QOS 时，可以按当前允许值改脚本，或向提交入口传入 sbatch 参数。先核对配置再执行；不要根据示例猜测自己的权限。

记录任务 ID。下面以 `12345` 作格式示例，换成自己的 ID：

Bash · 平台终端

```
scontrol show job 12345
tail -n 50 logs/compute-smoke-12345.out
tail -n 50 logs/compute-smoke-12345.err
```

开始计算后，日志应显示实际 `device=cuda`、GPU 名称、训练步骤和保存位置。CUDA 不可用时程序报错，不会悄悄用 CPU 假装 GPU 训练成功。

任务结束后若 `squeue` 看不到它，继续在网页历史记录查看结果；若平台支持 accounting，也可以查：

Bash · 平台终端

```
sacct -j 12345 --format=JobID,State,ExitCode,Elapsed
```

任务状态、退出结果、日志和产物一起判断。排队不是失败；资源限制或已有应用占用可能延迟分配。需要取消当前任务时执行 `scancel 12345`，只填写要取消的那个 ID。

步骤 3：检查一次训练产物

默认目录为 `outputs/smoke-12345/`，包含：

- `run.json`：实际设备、环境版本、配置、步骤数、状态和参数变化探针。
- `metrics.csv`：训练及验证指标、对应步骤和计算时间。
- `model.pt`：本次保存的模型参数。
- `loss.png`：结果曲线；一次短训练可能只有一个记录点。

在同一项目环境中读取：

Bash · 平台终端

```
"$COMPUTE_ENV_PREFIX/bin/python" inspect_run.py \
  outputs/smoke-12345 --require-cuda
```

脚本核对完成状态、CUDA 记录、模型可读取及指标与步骤对应。它是自学检查工具，不评分。`parameter_changed` 检查第一层权重探针经过更新，并不承诺短训练已经学到一个好分类器。

本章完成检查：有实际任务 ID 和完整日志，记录显示 `device=cuda`；结果检查能够读取模型，配置、完成步骤和指标相符。接下来先下载结果并确认资源结束。

2.3 取回结果，处理问题并结束任务

正常训练结束后，先取回日志和结果、确认资源已结束，再按需要阅读排错演示与速查表。演示中的错误任务只用于讲解。

2.3.1 下载结果、复用项目与释放资源

通过文件管理下载本次运行的结果目录及对应两份日志。需要打包时，在**平台项目根目录**执行，任务 ID 换成自己的：

Bash · 平台终端

```
tar -czf smoke-12345-result.tar.gz \
  outputs/smoke-12345 \
  logs/compute-smoke-12345.out logs/compute-smoke-12345.err
```

在 Windows 本地解压后可以预览 `run.json`、CSV 和图片。要用 `inspect_run.py` 读模型，需要本地也有兼容项目环境，具体见迁移选读附录。模型和完整数据作为自己的学习结果保留，本实验不要求提交。

批处理任务完成会结束资源分配；临时交互会话用 `exit`，网页应用应确认其“停止/取消”结果。关闭浏览器与停止应用是不同操作，实际行为在平台预演中核对。平台任务说明

下一次使用时，已有环境和数据通常可以复用。换配置使用新的输出目录，保存本次参数与环境版本。要观察更多 GPU 计算负载，继续同一项目的迁移与较大训练。

主线完成检查：本地已保存对应任务的结果目录和日志；批处理任务已结束，使用过的临时会话与网页应用也已确认停止。到这里可结束本次跟练，或继续下面的演示与附录选读。

2.3.2 演示：本地 Agent 辅助排错（可跳过）

下面是**主动设置的数据路径错误演示**。正常跟练可以跳过，学生无需制造错误或提交记录。实际错误与修复是否成功，要以自己的任务日志和重跑结果判断。

如果刚才已关闭平台终端，重新打开登录/提交终端，进入同一 `compute-practice/` 项目根目录，并按短 GPU 训练的提交步骤重新设置 `COMPUTE_ENV_PREFIX`，指向已核对的项目环境。随后在这个平台终端提交演示任务：

Bash · 平台终端

```
bash scripts/submit.sh path-error
```

错误任务让 `scripts/run-job.sh` 的 `path-error` 分支使用 `data-does-not-exist`。环境和 GPU 检查通过后，训练程序应报数据目录不存在。记下错误任务 ID，下载该任务的 `.out`、`.err`，同时在项目目录记录：

Bash · 平台终端

```
pwd
ls data/cifar-10-batches-py
```

把日志放入 **Windows 本地项目**的一个排错记录目录，让 agent 同时看到代码与日志。可以这样请求：

示例文本

```
这是平台任务的数据路径错误演示，请读取提交脚本和我下载的日志。
命令在平台执行，本地目录只是用于修改项目。
平台工作目录和实际数据目录见我提供的 pwd、ls 输出。
请解释失败位置，并只修正 path-error 分支的数据路径。
保留 GPU 要求和资源参数，给出修改差异及平台重跑步骤。
你未在平台执行的部分请标注待验证。
```

正常数据已确认位于项目的 `data/` 时，修复就是让这个分支引用实际数据目录，而非把设备改成 CPU、增大 GPU 数或重装依赖。先看 agent 的文件差异，再把修订后的 `scripts/run-job.sh` 上传覆盖演示副本。

再提交同一 profile，会得到新的任务 ID 和输出目录。检查新任务的 CUDA 记录、完成状态、日志与模型；这一步才是“修复在平台上有效”的证据。最后按结果下载与资源收尾保存演示的日志和结果，确认任务与临时应用已结束。

2.3.3 平台常见问题速查

现象	先检查什么
任务一直排队	等待原因、账号权限、已有运行任务和应用、申请资源是否过大
找不到 Python	`COMPUTE_ENV_PREFIX` 和其中的 `bin/python`；交互终端激活不会自动传给批处理
找不到 CIFAR-10	作业工作目录、`--data-dir`、目录结构和完整性校验
CUDA 不可用	是否实际分配 GPU、wheel 的 CUDA 构建、节点 GPU/驱动；保留错误再排查
提交内存超限	当前允许申请范围；与运行时 OOM 分开判断
`CUDA out of memory`	显存需求，先调整 batch 等计算配置；`--mem` 不是显存开关
没有日志	任务是否启动、提交前日志目录是否存在、日志路径与作业目录
输出目录已存在	选择新的运行目录，保留旧结果

向 AI 或助教求助时提供操作位置、任务 ID、脚本、日志、资源参数及必要目录信息。平台 FAQ、Slurm 速查

平台示例尚未完成普通学生账号的 GPU/Slurm 全流程预演；环境、资源申请与下载来源应按当前账号核对。需要帮助时，把实际步骤和日志交给 Agent 或助教。

附录 A：图像分类项目迁移与较大训练（选读）

本附录沿用短 GPU 训练中的 CIFAR-10 + ResNet-18 程序：先在 Windows 本地验证入口，让本地 Agent 检查迁移条件，随后在平台逐步扩大单卡训练，并把结果拿回本地。资源和训练配置仍以权限核对与预跑结果为准，不承诺固定加速倍数。

A.1 本地短测与迁移检查

步骤 1：在 Windows 本地做短检查

已有项目环境可先核对 torch 与 torchvision 的匹配；需要新环境时，使用独立环境，避免改动原课程环境。以下在本地 PowerShell 执行，环境放在项目外的 `.venvs/`：

PowerShell · Windows

```
python -m venv '..\.venvs\compute-cpu'
$computePython = '..\.venvs\compute-cpu\Scripts\python.exe'
& $computePython -m pip install torch==2.7.1 torchvision==0.22.1 `
  --index-url https://download.pytorch.org/whl/cpu
& $computePython -m pip install -r requirements.txt
```

使用普通 Python 的 venv 或你熟悉的独立 conda 环境均可，核对实际解释器；本地 CPU 检查不会证明平台 GPU 可用。

在本地项目中，按准备 CIFAR-10 中的方式下载并解包数据：

PowerShell · Windows

```
python prepare_data.py --archive '.\cifar-10-python.tar.gz' --data-dir '.\data'
& $computePython train.py --config configs/smoke.json `
  --device cpu --data-dir data --output outputs/local-check --max-steps 2
& $computePython inspect_run.py outputs/local-check
```

检查 `run.json`、指标、图片和模型能读取；已有目录不覆盖，再跑时换一个输出名称。CIFAR-10 的官方测试集始终不用于这里的调参。

步骤 2：让本地 Agent 检查迁移条件

把当前平台确认过的环境前缀、分区/QOS、GPU 申请写法、CPU、主内存和时限给 agent。让它读取 `configs/train.json`、`scripts/train.sbatch` 与目录结构，说明需修改的部分。

示例文本

```
请把这个本地已短测的 CIFAR-10 / ResNet-18 项目准备为平台单卡任务。
平台环境和资源规则以我提供的实际信息为准。
请检查数据路径、解释器、训练配置、日志和输出目录。
先解释修改理由，再修改必要的配置和提交脚本。
保持固定数据划分与种子，不自动扩大到多卡。
给出我在平台执行的命令和应检查的结果。
```

Windows 环境目录不用上传到 Linux；在平台按版本说明建立环境。只上传代码、配置和所需数据，已有平台数据可复用。

A.2 扩大训练与取回结果

步骤 1：逐步扩大，准备单卡任务

候选 `configs/train.json` 使用训练池的 45000 个样本、5000 个验证样本、batch 128、10 个 epoch，`max_steps=0` 表示不额外限制步骤。它用于展示持续计算，**尚未在平台证明能在候选时限内完成**。

可以先复制为自己的配置，依次增加样本数、训练步数和 epoch。先观察 CPU workers、显存和训练速度，再决定是否改 batch、资源或申请更长时限。

在已核对环境与资源的**平台项目根目录**：

Bash · 平台终端

```
export COMPUTE_ENV_PREFIX="$HOME/.conda/envs/compute-resnet"
bash scripts/submit.sh train
```

`scripts/train.sbatch` 的单卡、4 CPU、8G、1 小时只是候选起点。按当前权限与预跑需求调整，任务不需要的资源不申请。等待原因中的用户上限还可能与已有应用/任务有关。

步骤 2：下载与读取结果

记录新任务 ID，查看对应日志和 `outputs/train-任务 ID/`。确认任务状态、实际 CUDA 设备、模型可读取、指标与完成步骤一致。

下载结果后，在本地环境读取，例如任务 ID 为 `12346`：

PowerShell · Windows

```
& $computePython inspect_run.py outputs/train-12346 --require-cuda
```

这里把保存的模型加载到本地 CPU，`--require-cuda` 核对的是这份运行记录是否来自 CUDA 训练，不要求本地有 GPU。

短训练或一次较大训练都不保证较高准确率。图表记录的训练损失来自训练时的批次，验证损失来自每轮末尾的模型；它们的用途是观察流程与训练变化。

A.3 观察计算速度与资源需求

CPU/GPU 比较使用同一脚本、相同数据规模、batch 和步骤数，固定并记录线程/worker 设置。CPU 只测有限步数，不必先把较大任务完整跑完。

`run.json` 记录训练时间、实际处理图片数、吞吐和 GPU 分配内存峰值。训练计时包含数据加载与更新，经过 GPU 同步；小配置和首次运行的启动成本会影响比较，因此该值不是通用性能排名。PyTorch 计时指南

另外记录下载、上传、环境准备和排队时间。显存峰值是 PyTorch 张量分配的部分统计，不能等同于整个设备用量或 CPU 主内存。结合任务资源统计再判断瓶颈。

最后保存代码版本、配置、环境与本次产物。这样以后可以迁移自己的小项目，也能明确告诉 agent 哪一步需要帮助。

附录 B：配置管理与工具扩展（选读）

已有一套可用本地配置后，再按实际需要尝试这些工具。决策树与平台主线使用现有文件和终端工具即可。

B.1 用管理器维护多套配置

CC Switch 适合图形化添加与切换 API 提供方，也提供 MCP、Skills、提示词和用量管理。当前 DeepSeek 预设支持原生 Responses 直连；旧教程中的“必须启用本地转换路由”不一定适用。切换后重启目标客户端，再检查实际模型和一次工具调用。它采用 MIT 许可证。发布说明可用于核对版本变化。

Cockpit Tools 偏向多平台账号、配额、多实例与供应商管理，适合已有多个工具和项目的用户。源码公开，默认许可为 CC BY-NC-SA 4.0，带非商业限制。

管理器负责配置或本地路由，模型调用仍由实际 API 服务商提供和计费。先用主线得到一套可用配置，再按需要选择一个管理器；让多个工具轮流改写同一套配置会增加排查难度。切换前备份，密钥和服务地址始终配成一组。

B.2 Skills：保存重复任务的方法

Skill 通常组织一项任务的操作说明、脚本和资源。例如“按统一步骤检查一个 Python 小项目”，可以作为重复使用的技能。它是否被读取、如何触发和能用哪些工具，由运行框架决定。

拓展阅读：Codex Skills 官方说明。适合已经多次完成同类任务，想把方法复用起来的同学。先读结构与触发方式，再尝试写一个小技能。

B.3 MCP：接入工具与数据

MCP 是 Model Context Protocol，用来约定客户端与外部工具或资源服务怎样交流。例如一个服务可以提供资料搜索或数据库查询工具；harness 连接它后，把获准能力交给 agent 使用。服务本身的授权、模型的工具调用能力和客户端支持仍需分别确认。

拓展阅读：MCP 官方入门、Codex MCP 配置说明。适合已经需要连接额外数据源或工具的同学。本练习使用现有文件与终端工具即可，无需安装 MCP 服务。

B.4 Codex CLI 补充入口

如需命令行补充入口，在已经安装 Node.js 的 Windows 环境中按 Codex CLI 官方说明运行：

PowerShell · Windows

```
npm install -g @openai/codex
codex
```

CLI 与桌面客户端共享用户配置。安装命令仍需要下载 npm 包，网络能访问 API 并不代表所有安装源都可访问。先配置环境，再在练习目录启动；Codex 主线不要求学生使用虚拟机；DSH 试用另按附录 C 准备独立环境。

附录 C：DSH 的条件与配置（选读）

DSH（DeepSeek Harness）同样可以连接模型服务并操作本地文件。本实验主线使用 Codex；本附录只介绍 DSH 在独立实验环境中的试用。历史问题已有部分修复，但官方仍将它列为未经过安全审计的开发者预览。官方安全说明

C.1 准备环境，选择发行版

准备可恢复的独立 Windows 虚拟机，只放公开练习副本，不共享个人文档盘。使用专门的实验 Key，在服务端账户或项目控制用量；虚拟机保护宿主，不会自动保护其中的 Key。工作目录与 workspace-write 主要约束写入，不能据此认为 DSH 只能读取这个目录。

官方 Windows 安装入口见 DeepSeek Harness；学校图文指南采用 DSH Desktop 社区发行版。选定一种发行版，记录安装来源、桌面版本与内核版本。本附录下面的表单名称参考学校指南；不同发行版的菜单可能不同。

在实验虚拟机安装并启动选定客户端。社区版首次向导默认连接 DeepSeek 官网；使用学校 Key 时先选“稍后配置”，然后添加学校提供方。Codex 的 PowerShell 接入脚本用于修改 Codex 配置，不用于配置 DSH。

C.2 连接学校 API

在“设置 → 模型 → 添加自定义提供方”填写：

字段	学校服务示例
提供方 ID	`ustc`
显示名称	中科大公共平台
Base URL / API 地址	`https://api.llm.ustc.edu.cn/v1`
API 协议	`openai-completions`，即 Chat Completions
API Key	你申请的学校实验 Key
模型 ID	从当前项目授权模型列表复制准确 ID，保留后缀

保存后获取可用模型并选中。学校 Key 与学校网关配成一组；普通 Key 输入框直接填写 Key。若使用自己的 DeepSeek 官方 Key，按所选客户端的 DeepSeek 提供方说明配置，费用由对应官方账户承担。DSH 模型服务配置

C.3 查看权限，验证真实调用

保留审批与受限工作区模式；仅在本机使用，不开启远程访问或端口转发，也不增装第三方插件。使用官方模型服务时，查看“设置 → 常规”中的会话日志上传选项，按实际资料选择；学校自定义服务与官方服务的路由分别核对。会话日志说明

先选中刚添加的模型，发送一次仅回答文字的请求：

示例文本

```
请用三句话解释模型与 Harness 的区别。
这次只回答文字，不读取或修改文件。
```

收到回答后到服务平台查看对应项目的用量。随后选中一份新的决策树项目副本，按决策树协作练习先读取和讲解，再限定 T1 修改并运行自查；使用已确认的课程 Python，自己检查文件差异和输出。问答成功与文件工具成功分别确认。

C.4 保存结果，结束试用

保存需要的学习结果，关闭客户端并确认运行进程结束。仅为本次试用创建的临时 Key 可在对应服务平台撤销；保留常用 Key 的原有安排。需要重做时恢复虚拟机快照或重新复制练习副本。

这条路线用于了解通用接入方式，不意味着 DSH 的安全边界已经验收。遇到版本或权限问题，回到 Codex 主线即可完成本实验的文件练习。